

Problem 1: The Last Astronaut

```
START

Input n
Input k

Set currentk = 2
Set lastpos = 0

Create an array astroarray with n elements
Set every element of astroarray to 0

FOR i from 0 to n - 1

    IF k is less than 2 OR currentk is less than 2
        Set k = 2
    END IF

    Set remaining = n - i

    IF i equals 0
        IF k is divisible by remaining
            Set currentk = remaining
        ELSE
            Set currentk = k modulo remaining
        END IF
    ELSE
        IF currentk is divisible by remaining
            Set currentk = remaining
        ELSE
            Set currentk = currentk modulo remaining
        END IF
    END IF

    Set step = 0
    Set deadpos = 0

    WHILE step is less than currentk

        IF i equals 0
            Increase step by 1
        END IF

        IF astroarray[lastpos] equals 0
            Increase step by 1
            Set deadpos = lastpos
        END IF

        Move lastpos to the next position
        If lastpos reaches the end of the array,
            wrap around to position 0

    END WHILE

    Set astroarray[deadpos] to 1

    IF deadpos is even
        Increase currentk by 2
    ELSE
        Decrease currentk by 1
    END IF

    IF remaining is greater than 1
        Display the eliminated astronaut
    ELSE
        Display the winning astronaut
    END IF

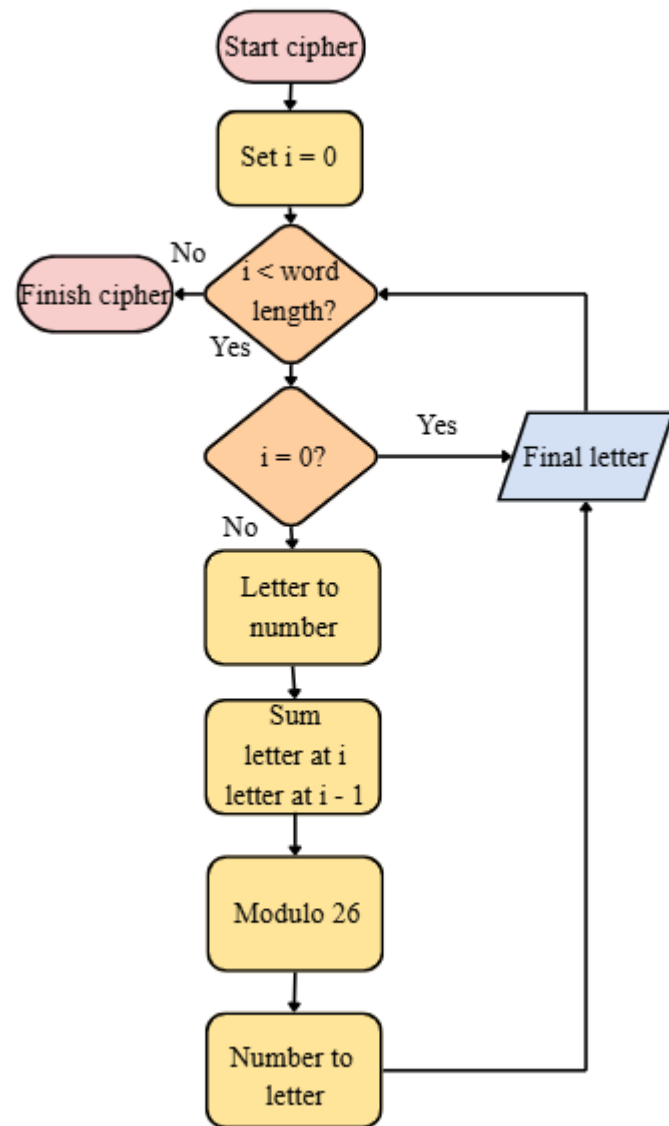
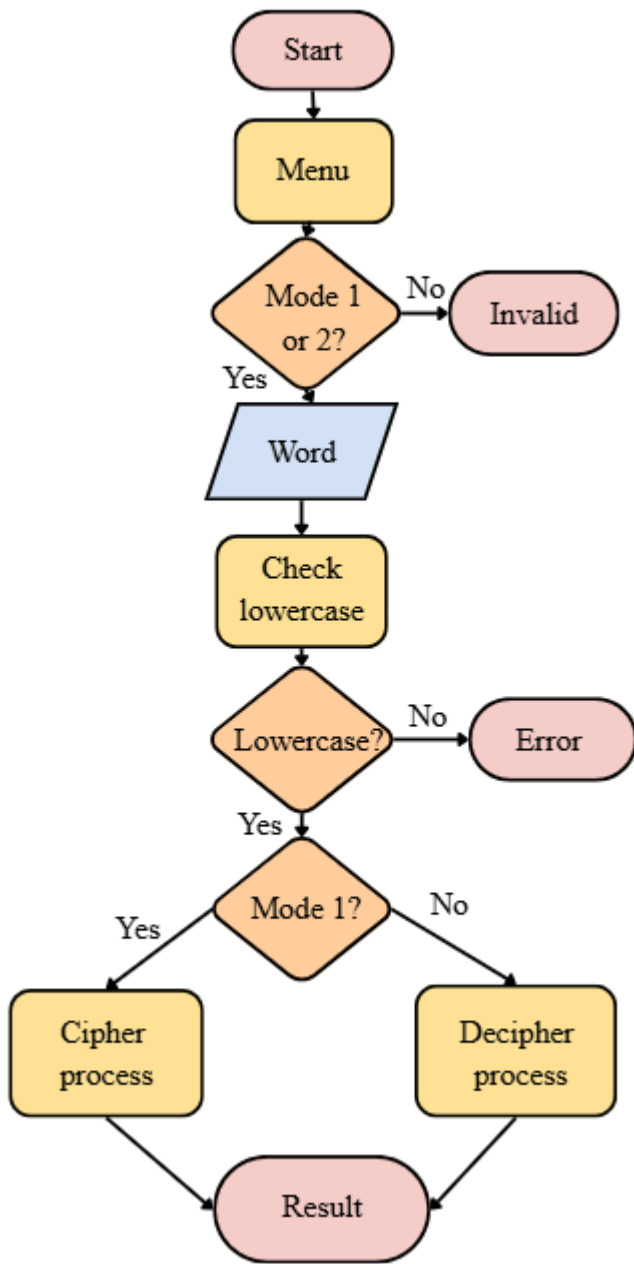
END FOR

END
```

This program is a twist on the classic Josephus Problem where n people stand in a circle numbered from 1 to n and are eliminated/executed one by one by a counting system, usually clockwise, k where the k th person is eliminated and counting resumes after the elimination. This problem just changes the counting system where k will increase by 2 if an even numbered person/astronaut is eliminated and k will decrease by 1 if an odd numbered astronaut is eliminated where 2 is the minimum value for k . This program was made in 2 parts, the first part was the Josephus Problem basis program and the second part was in modifying the elimination system.

Firstly, an input of ' n ', number of astronauts, and ' k ', elimination variable, is needed. Then an array of size n 'astroarray' to keep track of the astronauts is indexed to 0 meaning status alive. Setting "currentk" which will be the modified elimination variable with "remaining" being the remaining astronauts alive. This program is essentially a for-loop where each elimination is done at every iteration up until the last remaining astronaut which will be the survivor. "currentk" will be processed first. It will be processed differently from the first iteration because after the first iteration it'll need to be updated after every elimination, following the rules of the game. After that is the while-loop for the elimination process. The condition of this while loop is as long as step is less than "currentk" where step will increase by one after each while-iteration. This part checks whether an astronaut is dead or alive by checking the index of an array by not adding a step at an array if an astronaut is dead. This part also takes care of the wraparound in cases where the last position "lastpos" is bigger than the ' n ' itself by using a modulo. Then, when the while-loop stops, the astronaut at the last position will be eliminated by changing its index at "deadpos" which is also equal to "lastpos". Then "currentk" value will change depending on the astronaut it just eliminated by looking at "deadpos". At last it will output a message about the elimination if the remaining astronaut is more than one. If it's less than one, then it'll announce the last remaining astronaut.

Problem 1: Alien-In-The-Middle



This program is similar to a caesar cipher but the shift depends on the characters of the word itself. The cipher itself works by keeping the first letter and modifying the other letters by adding the second letter with the first letter, the third letter with the second letter, until the n th letter with the $n - 1$ th letter. The program works as a ciphering and deciphering device, where it applies the rule stated above for the ciphering and an inverse of that rule for the decipher, by keeping the first letter, and subtracting the second ciphered letter by the first real letter, the third ciphered letter by the second real letter, until the n th ciphered letter by the $n - 1$ th real letter. This program only works to cipher and decipher words in lowercase letters which is a bit inconvenient for the aliens that use this program because including the uppercase letters, symbols, or even sentences is far beyond my capabilities at this time.

The program starts by asking the user of the mode they're going to use, either cipher mode or decipher mode. Then it'll receive a word as an input which will be checked for and non-lowercase letters including symbols and numbers. It'll then process the word in either cipher or decipher mode respectively. In general, the program will convert the letters one by one, excluding the first one, via for-loop with the limit of 'i' is less than the length of the word. In cipher mode, at 'i' equals 1, which the program will start processing the letters, the program will take the 'i'th letter and add it with the 'i' - 1th letter and print it at every iteration. For the operations in the code itself, letters are first turned to integers and indexed as a = 1, b = 2, up to z = 26, which will be easier to compute. This is achieved by subtracting 'a' from the letters, adding 1 to them, and re-adding 'a' back at the end. Using the modulo function as a wraparound in cases where the sum of the letters exceeds 26. In the case of a letter having a value of 26 after an operation, which if modulo-ed and added to 'a', will result in the symbol ''. This is supposed to be the letter 'z', which explains why there is this specific "print the letter z" line of code in both the cipher mode and the decipher mode in the program. The decipher mode is just the inverse of the cipher mode as stated above.